OpenZeppelin | security

Dissolve The Guardians Audit



October 27, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 5

Security Model and Privileged Roles 6

Low Severity 7

L-01 Timelock Role Migration Strands Proposals In Flight On The Current Governor 7

L-02 Security Council Rotation Can Remove Guardians From The Emergency Path Before The Guardians Removal 7

L-03 Governance Signing Application Is Incompatible With the Upgraded Handler and Board 8

L-04 Proposals Created on Redeployed Governors Before Migration Escape the Veto 8

L-05 Superseded Upgrade Messages Remain Executable and Can Reopen the Initializer 9

L-06 Tests and CI Report Coverage the Guardians Removal Does Not Have 9

L-07 Two Interchangeable Redeployed Governor Pairs Are Live and None Is Pinned 10

Notes & Additional Information 11

N-01 Pre-v31 Guardians Retain the L2 Veto Until the Governor Migration Executes 11

Conclusion 12

Appendix 13

Issue Classification 13

Summary

Type	Governance
Timeline	From 2026-10-22 To 2026-10-23
Languages	Solidity

Total Issues	8 (3 resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	0 (0 resolved)
Low Severity Issues	7 (3 resolved)
Notes & Additional Information	1 (0 resolved)
Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed a diff audit of pull request #51 in the zksync-association/zk-governance repository, covering the changes between base commit `84c50f5` and head commit `fdbb05e`

In scope were the following added or modified files:

```
l1-contracts
└── src
    ├── interfaces
    │   └── IProtocolUpgradeHandler.sol
    ├── EmergencyUpgradeBoard.sol
    ├── ProtocolUpgradeHandler.sol
    ├── ProtocolUpgradeHandlerStateCheck.sol
    └── TestnetProtocolUpgradeHandler.sol
```

The diff also deletes the `Guardians` contract and its `IGuardians` and `IL2Governor` interfaces. The L1 deployment scripts, the L2 governor redeployment scripts and the test files were not in scope, although some issues in this report concern them.

System Overview

The `ProtocolUpgradeHandler` executes ZKsync protocol upgrades on L1. It moves each proposal from the L2 protocol governor through a legal veto period, an approval stage and an execution delay. Before this change, the Guardians multisig could extend the legal veto period and approve an upgrade without the Security Council. The Guardians also co-signed emergency upgrades with the Security Council and the ZK Foundation, and could propose or cancel proposals on the L2 governors through an L1 transaction.

The pull request dissolves the Guardians. Their role in the handler passes to the ZK Foundation, and their L1 path to propose or cancel L2 proposals is deleted without a replacement. Outside the scope, the pull request also adds scripts that redeploy the `ZkTokenGovernor` and `ZkGovOpsGovernor` with the ZK Foundation's Safe on Era as their veto guardian.

The in-scope changes are:

- **Protocol upgrade handler** (`ProtocolUpgradeHandler`): the `guardians` role becomes `zkFoundation`, together with its modifier, its approval function, its setter, its events and the fields of the upgrade status. The renamed variables keep their storage slots. The upgrade state machine and its periods are unchanged.
- **Emergency Upgrade Board** (`EmergencyUpgradeBoard`): an emergency upgrade now requires signatures from the Security Council and the ZK Foundation only.
- **State check** (`ProtocolUpgradeHandlerStateCheck`): a new stateless helper meant to be the first call of an upgrade proposal. It reverts unless the handler's Security Council and Emergency Upgrade Board match the expected addresses. A reverted execution leaves the proposal unexecuted, so a proposal can wait for a preceding upgrade that rotates these roles. The helper does not check the `zkFoundation` address.

Security Model and Privileged Roles

ZK Foundation (`zkFoundation`): The address is set by `initialize` or by `updateZkFoundation`, which only the handler itself can call as part of an executed upgrade. The ZK Foundation can extend the legal veto period of an upgrade once through `extendLegalVeto`. It can also approve a waiting upgrade through `approveUpgradeZkFoundation`. An upgrade approved this way becomes executable only after the 30-day wait period and the execution delay. The legal veto itself happens off-chain: the handler has no function to cancel an upgrade.

Security Council (`securityCouncil`): Its powers are unchanged. It can approve an upgrade without the 30-day wait period and can freeze and unfreeze the protocol.

Emergency Upgrade Board (`emergencyUpgradeBoard`): Through `executeEmergencyUpgrade`, it executes an upgrade at once, skipping the veto, approval and delay stages, and lifts any active freeze. After this change, the signatures of the Security Council and the ZK Foundation are enough to trigger it. If both are compromised or collude, they can execute any upgrade without delay. The board stores both signers as immutable values set at deployment. Rotating either role in the handler does not change who signs emergency upgrades; that takes a new board and a call to `updateEmergencyUpgradeBoard`.

The handler's `initialize` function is gated by `reinitializer(2)`, and on a proxy below that version any caller can invoke it. Its docstring states that the mainnet and testnet proxies are already initialized at this version, so their upgrades rotate roles through the `update*` functions instead. The ZK Foundation and the Security Council are assumed to act in the best interest of the protocol and its users.

Low Severity

L-01 Timelock Role Migration Strands Proposals In Flight On The Current Governor

The token governor migration and the GovOps governor migration revoke the proposer, canceller, and executor roles from the current governor without handling proposals in flight. Once the batch executes, an operation the current governor already queued cannot be executed, cannot be vetoed because the veto covers only pending and active proposals, and is unknown to the new governor. Proposals still pending, active, or succeeded on the current governor revert at queue. Passed proposals can therefore be lost, and a re-proposal with the same description collides with the stranded timelock operation until a further proposal cancels it.

Consider cancelling outstanding operations within the migration batch, or freezing proposals on the current governor and asserting at execution that the timelock holds no pending operations.

Update: *Acknowledged, not resolved. The team stated:*

> *We are acknowledging the issue. We will select a time for this protocol upgrade when the pending proposals queue is empty.*

L-02 Security Council Rotation Can Remove Guardians From The Emergency Path Before The Guardians Removal

The `RedeploySecurityCouncil` script reads only the Security Council, the Emergency Upgrade Board and the ZK Foundation Safe from the handler, and always deploys an `EmergencyUpgradeBoard` that checks only Security Council and ZK Foundation signatures. Unlike the removal script, it never checks that the handler has already dropped the Guardians. If a rotation built against a v31 handler is approved, emergency upgrades would no longer require Guardians signatures while the handler still relies on Guardians for standard upgrades. Such a rotation would also make a pending removal proposal revert at its Security Council check.

Consider requiring the post-removal handler, for example by requiring `zkFoundation` to succeed, before deploying the two-party board.

Update: *Resolved in pull request #54 at commit 40e7a81.*

L-03 Governance Signing Application Is Incompatible With the Upgraded Handler and Board

The signing application used by the Security Council and the ZK Foundation (`zksync-upgrade-verification-tool`) still reads `guardians` from the handler, a getter the upgraded handler no longer exposes. Role resolution fetches the Guardians member list alongside the other roles, so the revert can break sign-in for every visitor. The application also encodes the five-argument `executeEmergencyUpgrade` call, while the new board accepts four, and it offers no flow for the Foundation Safe to call `extendLegalVeto` or `approveUpgradeZkFoundation` directly. Until a compatible release ships, emergency upgrades and approvals cannot be assembled through the tool and require hand-built signatures and transactions.

Consider releasing an application update with the new handler and board interfaces and Safe-based Foundation flows before the upgrade executes.

Update: *Acknowledged, will resolve. The team stated:*

> *The new governance frontend will support the new signing format.*

L-04 Proposals Created on Redeployed Governors Before Migration Escape the Veto

The redeployed governors are deployed standalone and accept proposals before the migration grants them timelock roles. Any holder above the 21M ZK threshold can therefore propose on them now, and the vote can conclude while the migration proposal is still pending. Since the guardian veto applies only to Pending and Active proposals, a proposal that has already Succeeded can be queued and executed as soon as the migration executes, and nobody can cancel it. Because the new governors hold no assets yet, the Foundation and monitoring tooling are unlikely to be watching them. The migration tests only create proposals after migrating.

Consider monitoring the redeployed governors from deployment onward and confirming before the migration executes that the chosen governor has no open or succeeded proposals. Alternatively, consider deploying the token governor with proposals guarded until the migration completes.

Update: *Acknowledged, will resolve. The team stated:*

> *Acknowledged. We will monitor the redeployed governors from the moment they are deployed, and before executing the migration we will confirm that the selected governor has no pending, active, or succeeded proposals.*

L-05 Superseded Upgrade Messages Remain Executable and Can Reopen the Initializer

The `startUpgrade` function accepts any proven message from any caller, and an unstarted message never expires, so the superseded ZIP-16 messages remain executable. ZIP-16 stage 0 upgrades the handler without initializing it. If approved and executed before ZIP-18, it leaves `initialize` callable by anyone, who can then assign all three roles. After the removal, the ZK Foundation alone can approve the stale message, which may reinstall the Guardians once its pending ownerships are restored. The state check does not guard against prior proposals.

Consider starting each superseded message once ZIP-18 stage 0 has executed so that it expires unapproved, publishing superseded identifiers as do-not-sign to every approving party, and making this a standard step of each resubmission.

Update: *Acknowledged, will resolve. The team stated:*

> *We agree. We will start the ZIP-16 upgrade only after ZIP-18 has been executed. ZIP-16 will expire without signatures.*

L-06 Tests and CI Report Coverage the Guardians Removal Does Not Have

Several tests pass without checking what they claim. `test_RevertWhen_UpdateZkFoundationOnlySelf` calls `updateSecurityCouncil`, so the access control of `updateZkFoundation` has no negative test. Both mainnet fork suites use an `onlyMainnet` modifier that returns instead of skipping, so they report passes with no fork attached, and the ordering tests return silently once mainnet runs v31. The CI workflow runs no L1 tests and its push trigger names `main` while the default branch is

`master`. The "already extended" and "already approved" reverts are untested, and the scripts README asks for 12 council members while `SecurityCouncil` requires exactly 8.

Consider skipping explicitly with `vm.skip` when the fork precondition is absent, correcting the mislabeled test, adding an L1 test job to CI, and covering the untested revert paths and script outputs.

Update: *Resolved in pull request #54 at commit c49c11e.*

L-07 Two Interchangeable Redeployed Governor Pairs Are Live and None Is Pinned

Two `ZkTokenGovernor` and `ZkGovOpsGovernor` pairs are deployed on ZKsync Era with identical settings. Only the second pair (`0x1c52…caff`, `0xb998…3a8c`) is source-verified, and none of the four matches the governor hashes recorded in the repository, which come from a different `zksolc` version than the `governors-deploy` profile. The token governor's `NEW_GOVERNOR` resume path and its GovOps counterpart compare only getter values, never the bytecode, and the GovOps script does not check the type of `CURRENT_GOVERNOR`.

If the timelock role migration names the unverified pair, the token timelock and its treasury may be handed to a build that cannot be attested.

Consider recording the intended addresses in the repository, checking the deployed bytecode hash against the `governors-deploy` artifact, asserting the current governor's type, pinning one compiler version across build, hash file and deployment, and stating that the first pair is abandoned.

Update: *Resolved in pull request #54 at commit 685bf6f and at commit 649e825. The team stated:*

> *The latest deployment was built with zksolc 1.5.17, which is the reason for the `AllContractsHashes.json` mismatch. Updated the hashes using zksolc 1.5.17.*

Notes & Additional Information

N-01 Pre-v31 Guardians Retain the L2 Veto Until the Governor Migration Executes

The Era `ZkTokenGovernor` and `ZkGovOpsGovernor` hold the alias of the pre-v31 `Guardians` contract as their immutable `VETO_GUARDIAN`, although v31 replaces that contract on L1. It can therefore cancel the TPP and GAP that move the veto seat while they are Pending or Active, leaving the L2 removal dependent on signers without any L1 role.

Consider documenting this residual power in the removal runbook and coordinating with the pre-v31 Guardians signers before proposing the migration.

Update: *Acknowledged, not resolved. The team stated:*

> *Acknowledged. We are aware that the pre-v31 Guardians alias remains the immutable `VETO_GUARDIAN` on the Era governors and can cancel the TPP and GAP while they are Pending or Active.*

Conclusion

Pull request #51 dissolves the ZKsync Guardians and hands their role in the `ProtocolUpgradeHandler` to the ZK Foundation. Emergency upgrades now need the Security Council and the ZK Foundation only. OpenZeppelin audited the L1 contract changes of the pull request up to commit `fdbb05e`.

No critical, high or medium severity issues were identified. The contract changes are small. They rename the Guardians role in the handler, drop the Guardians signature from emergency upgrades, delete the Guardians contract and add a helper that checks the handler's roles. The handler's storage layout is preserved. Most of the reported issues concern the migration around these contracts, such as the order of its steps and the state that exists while it is in progress.

The Matter Labs team was responsive throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.